

REACT: LAYOUT, ROUTING Y ESTRUCTURA

Clase #09 - 01 de junio del 2026

Matías Corti - Martina García Améndola

TEMAS

- Salir del caos: estructura "profesional" del proyecto
 - Componente Layout (lo que se repite en todas las páginas)
 - React Router: navegar sin recargar la página
 - Estado con arrays y objetos: inmutabilidad
 - Empezar a separar la app en páginas reales
-

DE DÓNDE VENIMOS

La clase pasada vimos...

- JSX, Componentes, Props tipadas, useState
- Creamos un proyecto con Vite + React + TS
- Maquetamos la home de la librería con react-bootstrap
- Cada grupo hizo un componente

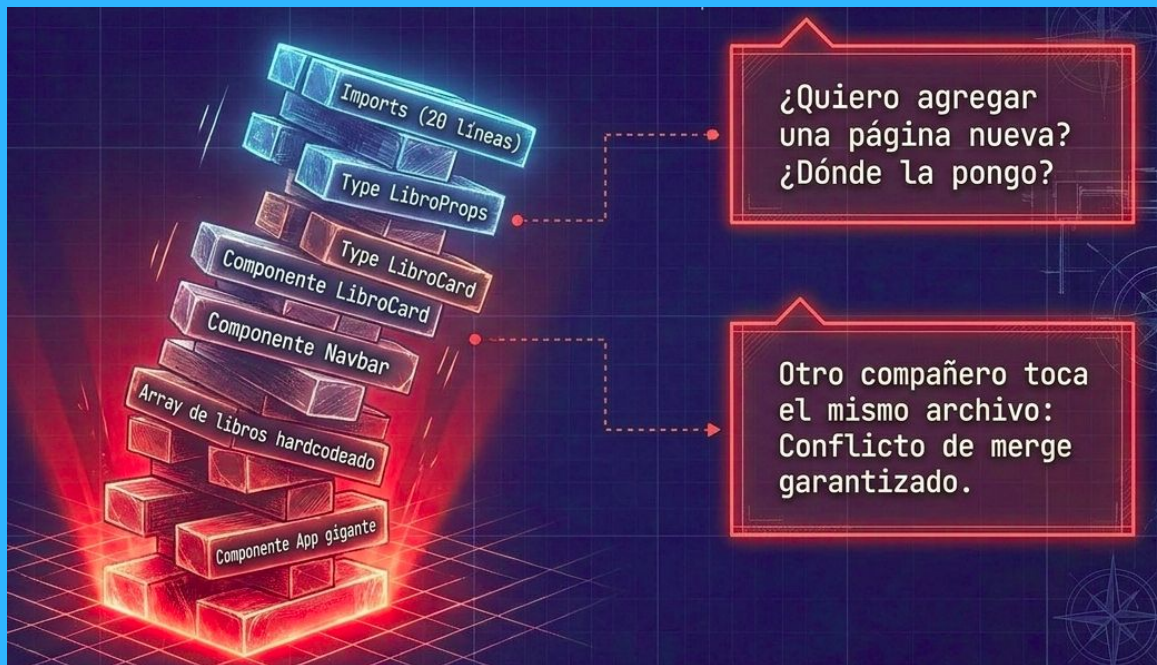
Hoy sumamos:

- Carpetas con criterio (no más App.tsx de 300 líneas)
- Layout que envuelve todo
- Navegación entre páginas (Router)
- Inmutabilidad para estado más complejo



HOY EN
APP.TSX...

APP.JSX
DESPUÉS DE
2 SEMANAS



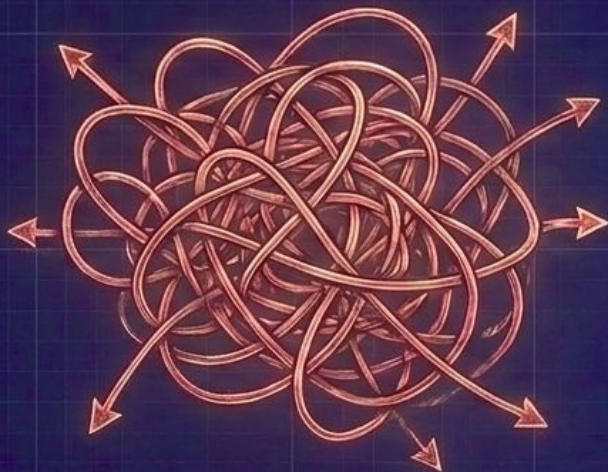
→ 300 líneas, todo mezclado

→ Quiero agregar una página nueva... ¿dónde la pongo?

→ Otro integrante toca el archivo → conflicto de merge garantizado

🎯 ¿PARA QUÉ REFACTORIZAR?

Enfoque Básico / Poco Profesional



Todo en un archivo. Rápido de empezar,
imposible de mantener en equipo.

App.tsx gigante, merge conflicts,
difícil reutilizar...

Estructura de Ingeniería Propuesta



🔧 Un componente = Un archivo = Una responsabilidad

Cada cosa en su lugar, archivos
distintos, **convención clara!**



ESTRUCTURA PROFESIONAL

¡Divide y vencerás!

[Repositorio de Ejemplo:](#)
[DS26-Libreria](#)

```
mi-proyecto/  
├── public/           ← Archivos estáticos  
├── src/              ← Nuestro código  
│   ├── assets/      ← Imágenes, CSS  
│   ├── components/  ← Componentes reutilizables  
│   │   ├── Layout/  ← Componentes del Layout  
│   │   │   ├── Header.tsx  
│   │   │   ├── Footer.tsx  
│   │   │   └── Layout.tsx  
│   │   ├── LibroCard.tsx  
│   │   └── SearchBar.tsx  
│   ├── pages/        ← Páginas completas  
│   │   ├── Home.tsx  
│   │   ├── LibrosCatalogo.tsx  
│   │   └── Contacto.tsx  
│   ├── types/         ← types/interfaces compartidos  
│   │   └── LibroCardProps.ts  
│   ├── App.tsx        ← Router principal  
│   └── main.tsx       ← Punto de entrada  
├── index.html         ← Index HTML donde se inyecta la App React  
├── package.json       ← Dependencias configuradas  
└── node_modules/      ← Código fuente de dependencias instaladas
```

 Existen **muchas maneras** de organizar los proyectos, lo fundamental es tener una **convención clara**



IMPORT EXPORT

 1 componente



∞ usos con datos
diferentes

EXPORTAR UN COMPONENTE

```
// src/components/LibroCard.tsx

import Card from 'react-bootstrap/Card';
import type { LibroCardProps } from '../types/LibroCardProps';

function LibroCard({ titulo, autor, imagen }: LibroCardProps) {
  return (
    <Card style={{ width: '18rem' }}>
      <Card.Img variant="top" src={imagen} />
      <Card.Body>
        <Card.Title>{titulo}</Card.Title>
        <Card.Text>{autor}</Card.Text>
      </Card.Body>
    </Card>
  );
}

export default LibroCard;  {/* ← Así lo exportamos */}
```

IMPORTAR Y USAR

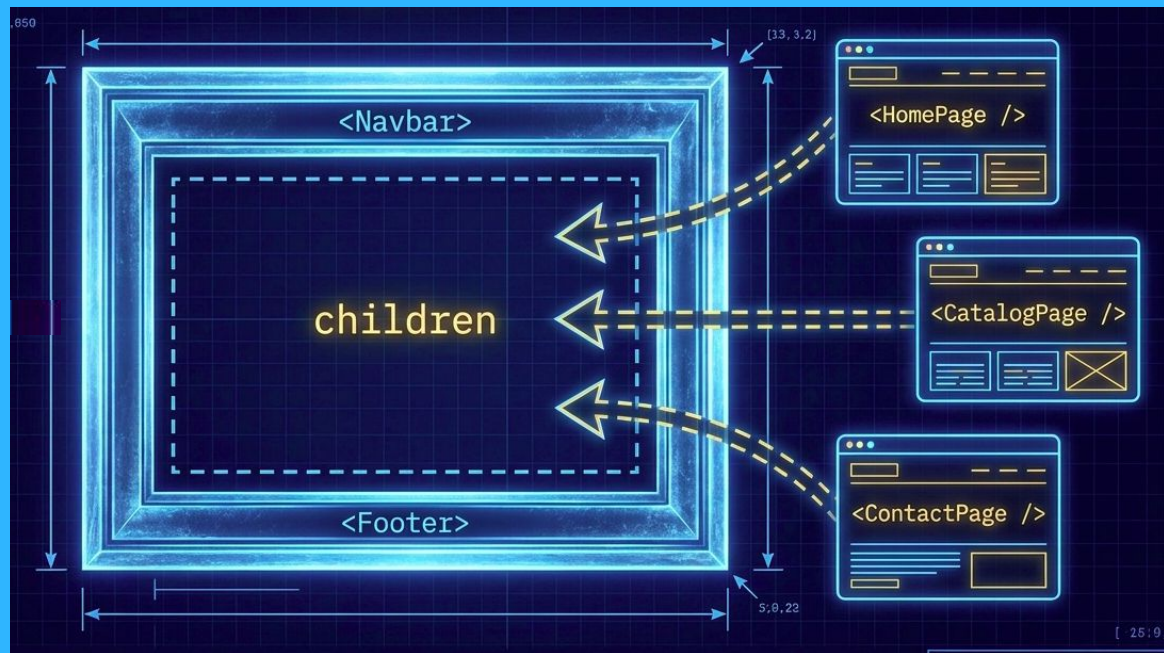
```
// src/pages/Home.tsx

import LibroCard from '../components/LibroCard';

function Home() {
  return (
    <div>
      <h2>Libros Destacados</h2>
      <LibroCard titulo="1984" autor="Orwell" imagen="..." />
      <LibroCard titulo="Cien Años..." autor="García Márquez" imagen="..." />
    </div>
  );
}
```




COMPONENTE LAYOUT: EL PATRÓN



Layout → Estructura de diseño reutilizable. Define las zonas fijas de la interfaz.

children → Prop especial que representa "todo lo que pones entre las etiquetas del componente".



LAYOUT CON TS Y REACT-BOOTSTRAP

```
// src/components/Layout/Layout.tsx

import type { ReactNode } from 'react';
import { Container } from 'react-bootstrap';
import Header from './Header';
import Footer from './Footer';
import '../styles/Layout.css';

interface LayoutProps { children: ReactNode };

function Layout({ children }: LayoutProps) {
  return (
    <div className="layout">
      <Header />
      <Container className="py-4">{children}</Container>
      <Footer />
    </div>
  );
}

export default Layout;
```

```
// src/App.tsx

import Layout from './components/Layout/Layout';
import Home from './components/Home';

function App() {
  return (
    <Layout>
      <Home />
    </Layout>
  );
}

export default App;
```



REACT ROUTER:

EL PROBLEMA QUE RESUELVE



REACT ROUTER

URL	→	Componente que se renderiza
/	→	Home
/catalogo	→	Página de Catalogo
/libros/42	→	Detalle del libro 42 (id = 42)
/libros/123	→	Detalle del libro 123 (id = 123)

Sin Router:

- ❌ La URL nunca cambia
- ❌ No se puede compartir un link a una página
- ❌ El botón "atrás" no funciona

Con Router:

- ✅ Cada URL muestra un componente distinto
- ✅ Navegación sin recargar la página (SPA real)



URL ↔
COMPONENTE

URL	→	Componente que se renderiza
-----	---	-----------------------------

/	→	<Home />
/catalogo	→	<LibrosCatalogo />
/libros/42	→	<LibroDetalle /> (id = 42)
/libros/123	→	<LibroDetalle /> (id = 123)
/cualquier-cosa	→	<NotFoundPage />

Router "escucha" la URL.

Cuando cambia, "decide" qué componente mostrar dentro del Layout.



REACT ROUTER NAVEGACIÓN REAL

IMPLEMENTACIÓN MANUAL

tsx

```
const [page, setPage] = useState<string>('home');  
  
{page === 'home' && <Home />}  
{page === 'catalogo' && <LibrosCatalogo />}
```



PROBLEMA: URLS NO CAMBIAN, NO HAY BOTÓN "ATRÁS"

REACT ROUTER

tsx

```
<Routes>  
  <Route path="/" element={<Home />} />  
  <Route path="/catalogo" element={<LibrosCatalogo />} />  
  <Route path="/contacto" element={<Contacto />} />  
</Routes>
```



SOLUCIÓN: CON REACT ROUTER SE ACTUALIZAN LAS URLS



INSTALACIÓN DE REACT ROUTER

PASO 1: INSTALAR

```
bash
```

```
npm install react-router-dom
```

PASO 2: CONFIGURAR MAIN.TSX

```
// src/main.tsx (se deja así una vez, no se toca más)
```

```
import { BrowserRouter } from 'react-router-dom';
```

```
import { StrictMode } from 'react'
```

```
import { createRoot } from 'react-dom/client'
```

```
import App from './App.tsx'
```

```
import './index.css'
```

```
createRoot(document.getElementById('root')!).render(
```

```
  <StrictMode>
```

```
    <BrowserRouter>      {/* BrowserRouter envuelve toda la App */}
```

```
      <App />
```

```
    </BrowserRouter>
```

```
  </StrictMode>,
```

```
)
```




INSTALACIÓN DE REACT ROUTER

PASO 3: CONFIGURAR APP.TSX

```
// src/App.tsx - queda así de reducido

import { Routes, Route } from 'react-router-dom';
import { Layout } from '../components/Layout/Layout';
import { Home } from '../pages/Home';
import { LibrosCatalogo } from '../pages/LibrosCatalogo';
import { Contacto } from '../pages/Contacto';
import { LibroDetalle } from '../pages/LibroDetalle';

function App() {
  return (
    <Layout>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/catalogo" element={<LibrosCatalogo />} />
        <Route path="/contacto" element={<Contacto />} />
        <Route path="/libros/:id" element={<LibroDetalle />} />
      </Routes>
    </Layout>
  );
}

export default App;
```



BROWSERROUTER → APP → LAYOUT → ROUTES → ROUTE



NAVEGACIÓN CON LINK

EN LUGAR DE `<a>` USAMOS `<Link>`

tsx

```
// ❌ MAL: <a> recarga toda la página (perdés todo el estado)
<a href="/catalogo">Ver catálogo</a>

// ✅ BIEN: <Link> cambia la URL sin recargar
<Link to="/catalogo">Ver catálogo</Link>

// ✅ Nav con Bootstrap: <Nav.Link>
<Nav.Link as={Link} to="/catalogo">Ver catálogo</Nav.Link>
```

¿Por qué importa?

`<a>` → el navegador hace un GET nuevo, React se reinicia

`<Link>` → Router intercepta el click, cambia la URL, muestra el componente nuevo. Sin recargar.

Regla: dentro de la app, SIEMPRE `<Link>`.

`<a>` solo para links externos (https://otro-sitio).



VELOCIDAD: CAMBIA CONTENIDO, NO RECARGA PÁGINA



USEPARAMS: LEER LA URL

```
// src/pages/LibroDetalle.tsx

import { useParams } from 'react-router-dom';

export function LibroDetalle() {
  const { id } = useParams<{ id: string }>();
  //      ↑ matchea con :id de /libros/:id
  return <h1>Detalle del libro {id}</h1>;
}
```

Flujo completo:

Usuario hace click en `<Link to="/libros/42">`

→ URL cambia a `/libros/42`

→ Router matchea `"/libros/:id"`

→ Renderiza `LibroDetalle`

→ `useParams` lee el **42** de la URL



ESTADO CON ARRAYS Y OBJETOS



ESTADOS COMPLEJOS: EL PROBLEMA

tsx

```
const [libros, setLibros] = useState<Libro[]>(librosIniciales);

// 🚨 ¿Qué pasa con esto?
function agregarLibro(nuevo: Libro) {
  libros.push(nuevo); // agrega al array..
  setLibros(libros); // pero React NO re-renderiza
}
```

React compara por REFERENCIA, no el contenido:

`libros === libros` → `true` → "no hay cambio" → no renderiza

Si es mismo objeto en memoria, asume que no cambió nada.

👁️ Es un error común al usar `useState`



INMUTABILIDAD DE ARRAYS

```
// ✓ Agregar: spread + nuevo elemento
setLibros ([...libros, nuevo]);

// ✓ Quitar: filter (devuelve array nuevo)
setLibros (libros.filter(libro => libro.id !== idABorrar));

// ✓ Actualizar uno: map
setLibros (libros.map(libro =>
  libro.id === idAEditar ? { ...libro, precio: 9999 } : libro
));
```

Regla:

NUNCA mutar el estado original.

SIEMPRE devolver un array/objeto **NUEVO**.

push, pop, splice → mutan ✗

...spread, filter, map → devuelven nuevo ✓



INMUTABILIDAD DE OBJETOS

```
const [usuario, setUsuario] = useState({
  nombre: 'Ada',
  email: 'ada@utn.edu.ar'
});

// ✅ Cambiar un campo: spread del objeto + override
setUsuario({ ...usuario, email: 'nuevo@utn.edu.ar' });

// ❌ NO HACER ESTO
usuario.email = 'nuevo@utn.edu.ar';
setUsuario(usuario); // misma referencia → no renderiza
```

Mismo patrón que con arrays:

"copio todo lo viejo y piso lo que quiero cambiar"



PRÁCTICA: 3 HITOS



PRÁCTICA:

3 HITOS

Vamos a refactorizar el ejercicio individual de C08 en 3 hitos. Cada uno termina con algo funcionando.

Hito 1 (20 min) → Mover componentes a carpetas

Hito 2 (15 min) → Layout con Navbar

Hito 3 (25 min) → Router con páginas navegables

✓ Cada hito tiene su commit

✓ El [branch C09](#) del repo de ejemplo es la guía

✓ Si se traban, levantan la mano y los ayudamos



HITO 1: REFACTOR A CARPETAS

 **Objetivo:** que App.tsx tenga menos de 30 líneas

Copiar el sitio de la librería de la Clase 08 en subdirectorío nuevo para esta clase:

`ejercicios/c09-libreria-react-layout/`

Paso a paso

1. Crear estructura de carpetas: `src/components/`, `src/components/Layout/`, `src/pages/`, `src/types/`

2. Mover el type Libro a `src/types/libro.ts`

3. Mover LibroCard a `src/components/LibroCard.tsx`

4. Crear `src/pages/Home.tsx`

→ copiar adentro lo que hoy está en `App()` (la home actual)

5. En `App.tsx`: importar y renderizar `<Home />`

6. `npm run dev` → debería verse IGUAL que antes

7. `git commit -m "refactor: estructura de carpetas"`

 Si se ve igual y `App.tsx` quedó reducido, hito hecho



HITO 2: LAYOUT CON NAVBAR

 **Objetivo:** Navbar y Footer que aparezcan en toda la app

Paso a paso

1. Crear los componentes Layout, Header y Footer
2. Layout debe incorporar a Header, Footer y a {children} entre los dos
3. Para el Header utilizar <Navbar> de react-bootstrap
4. En App.tsx: envolver <Home /> con <Layout>
5. npm run dev → ahora hay Navbar arriba y Footer abajo
6. git commit -m "feat: agregar Layout"

 Si ves la Navbar arriba en la home, hito hecho



HITO 3: ROUTER + 3 PÁGINAS

 **Objetivo:** páginas navegables

Paso a paso

1. Importar { Routes, Route } en App.tsx
2. Crear:
 - src/pages/Catalogo.tsx (lista de libros)
 - src/pages/LibroDetalle.tsx (usa useParams)
3. Definir las rutas en App.tsx (3 <Route>)
4. En Layout, los <Nav.Link> usar as={Link} to="..."
5. En LibroCard, el botón "Ver más" lleva a /libros/{id}
 - usar <Link to={`/\${libros}/\${libro.id}`}>
6. npm run dev → navegar entre las 3 páginas SIN recargar
7. git commit -m "feat: routing con páginas"

 Si la URL cambia sin recargar, hito hecho!



ANTES DE LA PRÓXIMA CLASE



INSTALAR ANTES DE C10

El aula no tiene internet confiable.

Sin instalación = sin práctica.

```
bash
```

```
npm zod react-hook-form @hookform/resolvers
```



Verificar con: `npm run dev`



Dudas → canal #consultas de Discord



C10: Formularios + Zod (08/06/2026)

ACTIVIDADES

ACTIVIDAD INDIVIDUAL DE LA SEMANA

1° instancia hasta el
07/06/2026
2° instancia: 05/07/2026
(Cierre Bloque 2)

[AI-C09 - Layout en Librería]

Sumar Layout y Navegación a la librería React

- En el repositorio ds2026-actividades copiar el sitio de la librería de la Clase 08 en subdirectorío nuevo para esta clase: **`ejercicios/c09-libreria-react-layout/`**
- Completar los 3 hitos de la practica.

Subir al repositorio y comunicar la finalización en Discord en un mensaje único por equipo, dentro del canal **#actividades**, en el Hilo:

"[AI-C09] Layout en Librería".

ACTIVIDAD GRUPAL

Hasta el **07/06/2026**
(inclusive)

[AG-C09 - Layout + navegación del TP]

1. Crear la estructura de directorios vista en clase en el proyecto frontend de repositorio del Grupo
 2. Implementar Layout con Header (incluye Navbar), Footer y children.
 3. Incorporar Router para la navegación principal
 4. Cada integrante debe maquetar al menos 1 página propia (pueden no ser completamente funcionales)
 5. Hacer PRs cruzados (al menos uno cada alumno)
 6. Trello: mover tarjetas correspondientes a "Done".
 7. Comunicar la finalización en Discord en un mensaje único por equipo, dentro del canal **#actividades**, en el Hilo: ***"[AG-C09] Layout + navegación del TP"***.
-